the whole list of points. Instead, we can use all the info we have to prune the search: in the geometric representation on the right, we show a visual hint of why it would be useless to check A's right sub-branch. If you check back figure 9.21, you can notice that, instead, we knew that we couldn't rule out D's right sub-branch without traversing it.

At node A we took the left branch during search, meaning P is on the left semi-plane. If we were to traverse the right subtree of A, all the points in that subtree would have their x coordinate greater than or equal to A's. Therefore, the minimum distance between P and any point in the right sub-tree of A is at least the distance between P and its projection on the vertical line passing through A (i.e. A's split line): in other words, the minimum distance for any point right of A will at least be the absolute value of the difference between P's and A's x coordinates.

We can therefore prune search on A's right branch, and since it was the root, we are finally done. Notice how the more we climb back on the tree, the largest are the branches (and the regions) that we can prune – and so the largest is the saving.

We can generalize this method to find an arbitrary large set of the closest points in the dataset, also known as the n-nearest-neighbor[11] search method.

The only differences are:

- Instead of the distance of a single point, we need to keep track of the m shortest distances, if we want the m closest points;